Universidad Camilo José Cela

Escuela Politécnica Superior de Tecnología y Ciencia

Desarrollo de Servidor y Big Data (76036)

Curso 2026/2027 · 1.º cuatrimestre

Práctica 2 Big Data

Del ejemplo al universo: países, serie histórica, Parquet y Spark

Práctica 2 Big Data

César Andrés Sánchez

Bloque II · Data Mining · Práctica 2 Big Data · v2.17.0

Identificación

Estudiante: _____ Grupo: _____

Fecha: _____ Nota: _____ / —

Contenidos originales bajo CC BY-SA 4.0: atribuir la autoría y compartir adaptaciones bajo la misma licencia o una compatible. Materiales de terceros, marcas y logotipos institucionales quedan excluidos salvo indicación expresa.
https://creativecommons.org/licenses/by-sa/4.0/

Cómo citar: César Andrés Sánchez. (2026). *Práctica 2 Big Data*. Desarrollo de Servidor y Big Data. v2.17.0. CC BY-SA 4.0.

Índice

Contexto **3**

Prototipos **3**

Continuáis en el GitHub de la Práctica 1 **4**

De dónde partís (lo que ya debe estar en ese repo) **5**

Qué tenéis que añadir al repositorio **5**

Ficheros y carpetas nuevos 5

Cambios obligatorios sobre lo que ya teníais 6

Qué tiene que hacer `src/transform.py` 6

Los seis indicadores del Banco Mundial (ejemplos reales) 8

Mapa mental: de lo viejo a lo nuevo 9

Teoría 1 · Por qué cinco países no bastan **9**

Teoría 2 · Paginación: la API os dice cuánto hay **10**

Qué es eso de la "predicción" de filas (y dónde va) 10

Teoría 3 · Extracción cortés e idempotente **12**

Qué hacer con cada código HTTP 12

Idempotencia 12

Teoría 4 · ¿Qué es un país? **14**

Teoría 5 · De fila a columna: por qué Parquet **15**

Qué quiere decir eso, en orden 15

CSV frente a Parquet 17

Anatomía de un fichero Parquet 19

Particionado estilo Hive 20

Teoría 6 · Tipado: el contrato que el CSV no impone **22**

 CC BY-SA 4.0

Teoría 7 · MapReduce **22**

Teoría 8 · Spark: el mismo modelo en vuestro portátil **24**

Driver y executors 24

Tres APIs para la misma cuenta 25

Transformaciones, acciones y pereza 25

Jobs, stages, tareas y particiones 26

Arranque de sesión (único fragmento de código que se os da) 26

Instalar Java para Spark (Linux y Windows) 27

Por qué Spark si el dataset aún cabe en memoria 28

Qué tenéis que conseguir en código **28**

Calendario y qué hay que entregar **29**

Referencias **32**

 CC BY-SA 4.0

Contexto

En la Práctica 1 construisteis un observatorio país con un ETL explícito (`raw` → `clean` → `serving`), tres fuentes abiertas, cinco países fijados a mano y un año por indicador. El producto era una ficha automática y un informe con tres hallazgos. Eso demuestra que el patrón está entendido. No demuestra que aguante datos reales.

Con cinco países y un año, una sola petición suele devolver **todo** lo pedido: la API no os obliga a pedir la página 2, la 3, etc. Eso es **paginar**: recorrer trozos porque el servidor no manda el universo entero de una vez. Al pedir todos los países y toda la serie, sí os fuerza a paginar, y a comprobar que lo guardado cuadra con el `total` que declara.

Esta práctica **no parte de un repositorio nuevo**. Continuáis en el **mismo GitHub del equipo** que usasteis en la Práctica 1: mismos `src/`, mismo `sources.yaml`, mismo pipeline. Lo que hacéis es *ampliar* ese código para que el observatorio cubra todos los países, toda la serie histórica, Parquet particionado y un primer job de Spark.

No se os reparte un starter nuevo. El enunciado fija qué hay que conseguir; cómo lo programáis es cosa vuestra. Antes de tocar código, leéis la teoría de las secciones siguientes: está escrita para que podáis decidir con criterio.

Parquet, Hive, MapReduce y Spark se ensayan primero en el kit `practica02_prototipos/` (o el ZIP `practica02_prototipos.zip`) (Prototipos), sin tocar aún el repositorio del equipo.

Prototipos

En el material del curso se reparte `practica02_prototipos.zip` (carpeta `practica02_prototipos/`): nueve scripts numerados, un CSV reducido de países/indicadores/años, `requirements.txt` y `README.md`. No forma parte del entregable ni sustituye a `spark/job_01.py`; se usa para comprobar el apilado técnico antes de aplicarlo al observatorio.

 CC BY-SA 4.0

Script	**Contenido / encaje en el repo**
`01_csv_a_parquet.py`	CSV → Parquet tipado (`transform.py`)
`02_leer_parquet.py`	Lectura Parquet → SQLite (`load.py`)
`03_particionado_hive.py`	Carpetas `indicator_id=.../` (`data/clean/indicators/`)
`04_consulta_pyarrow.py`	Dataset Hive y filtro con poda
`05_mapreduce_puro.py`	Map→Shuffle→Reduce en Python (Teoría 7)
`06_spark_session.py`	`SparkSession` en `local[*]`
`07_spark_mapreduce_rdd.py`	RDD `map`/`reduceByKey` (tramo RDD de `job_01`)
`08_spark_dataframe_sql.py`	DataFrame, SQL y salida Parquet de cobertura
`09_csv_vs_parquet_tamano.py`	Comparación de tamaño CSV frente a Parquet

Desde esa carpeta:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
./run_all.sh
```

Los scripts 06–08 requieren Java 17, 21 o 25. Los artefactos se escriben en `out/` (fuera de Git). El mismo esquema se aplica después a los datos del repositorio del equipo.

Continuáis en el GitHub de la Práctica 1

Un solo repositorio por equipo. El de la Práctica 1. No creéis otro "para la 2" y abandonéis el primero: la historia de git (predicción, arreglos, no regresión) se evalúa ahí. La figura 1 resume ese hilo.

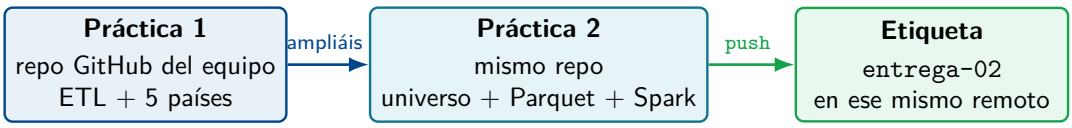


Figura 1: Continuidad del repositorio: de la Práctica 1 a la etiqueta `entrega-02`.

- Si el repo es privado, el colaborador `cesar-andress` ya debería tener lectura desde la Práctica 1; si no, añadidlo **antes** de la fecha límite.
- Trabajad en `main` (o `master`).

 CC BY-SA 4.0

- Al cerrar: `git tag -a entrega-02 -m "Práctica 2"` y `git push origin entrega-02`.

De dónde partís (lo que ya debe estar en ese repo)

Antes de escalar, el clon limpio de vuestro GitHub de la Práctica 1 debe regenerar:

- `python -m src.pipeline` con venv y `requirements.txt`.
- Zonas `data/raw/`, `data/clean/`, `data/serving/`.
- `sources.yaml` con fuentes activas y `focus_countries`.
- Transformaciones del caso pequeño (ISO3, idiomas largos, esquema común, tipado, `join_orphans.csv`).
- SQLite y fichas en `cards/` generadas por código.

Si eso no regenera, arregladlo primero. Esta práctica **extiende** la anterior; no la sustituye. Lo que hacía la Práctica 1 debe seguir funcionando (no regresión).

Qué tenéis que añadir al repositorio

Todo esto vive **dentro del mismo repo**. La lista es el contrato de ampliación: si falta un ítem, la práctica no está completa.

Ficheros y carpetas nuevos

```
spark/job_01.py                 # job Spark (RDD + DataFrame + SQL)
CONTRIBUTORS.md                 # integrantes: nombre, apellidos, email
    ↪ UCJC
ENTREGA_02.md                   # predicción, resultado, decisiones, teoría
BUGS.md                         # >= 3 defectos que el universo destapa
evidencias/                     # subdirectorio del event log citado
data/clean/countries.parquet
data/clean/languages.parquet
data/clean/excluded_entities.csv
data/clean/join_orphans.csv     # ahora: huérfanos por fuente
data/clean/indicators/indicator_id=<id>/*.parquet
data/serving/spark/cobertura_decada/*.parquet
logs/spark-events/              # se regenera; no se sube (sí evidencias/)
cards/<ISO3>.md                 # una ficha por cada país aceptado
```

 CC BY-SA 4.0

Cambios obligatorios sobre lo que ya teníais

`sources.yaml`
Universo completo (ya no basta filtrar solo `focus_countries` en el perfil). Al menos seis indicadores del Banco Mundial (véase la tabla de ejemplos más abajo). Criterio de país y equivalencias de códigos en configuración, no en un `if` suelto.

`src/extract.py` (o equivalente)
Paginación completa contra el total de la API; no guardar como dato nada que no sea HTTP 200; reintentos con espera creciente ante 429/5xx/timeouts; idempotencia sin una opción explícita de forzar re-descarga; manifiesto **acumulativo** con `run_id`; máximo 5 peticiones/s y `User-Agent` del equipo.

`src/transform.py`
Sigue siendo el paso que lee `data/raw/` y escribe `data/clean/`, pero ahora a escala de universo: criterio de país, tipado, Parquet y huérfanos por fuente (detalle y diagrama en la subsección siguiente).

`src/load.py` / escritura clean
`clean/` en Parquet; `indicators/` particionado estilo Hive por `indicator_id`; SQLite de `serving/` regenerado desde ese limpio.

`src/build_cards.py`
Una ficha por país aceptado; comparativo con la mediana de la región en el **mismo** indicador y el **mismo** año.

`requirements.txt`
Añadir lo que uséis (`pyarrow`, `pyspark`, ...). Si alguien lo instaló a mano y no está aquí, el clon limpio no regenera.

`.gitignore`
Debe ignorar `data/`, `logs/`, `.venv/`. No subáis volcados brutos ni el event log completo: solo el trozo citado, en `evidencias/`.

Qué tiene que hacer `src/transform.py`

En la Práctica 1 este módulo ya existía: leía el JSON de `data/raw/`, filtraba a cinco países, aplanaba el perfil, explotaba idiomas y normalizaba indicadores a un esquema común. Eso **no se tira**. En esta práctica lo ampliáis para que el mismo módulo aguante el universo y deje la zona limpia en Parquet tipado.

El Transform **no descarga** nada y **no escribe** el SQLite: solo convierte bruto en limpio. La figura 2 ordena esas responsabilidades de arriba abajo.

Paso a paso, qué implica cada caja:

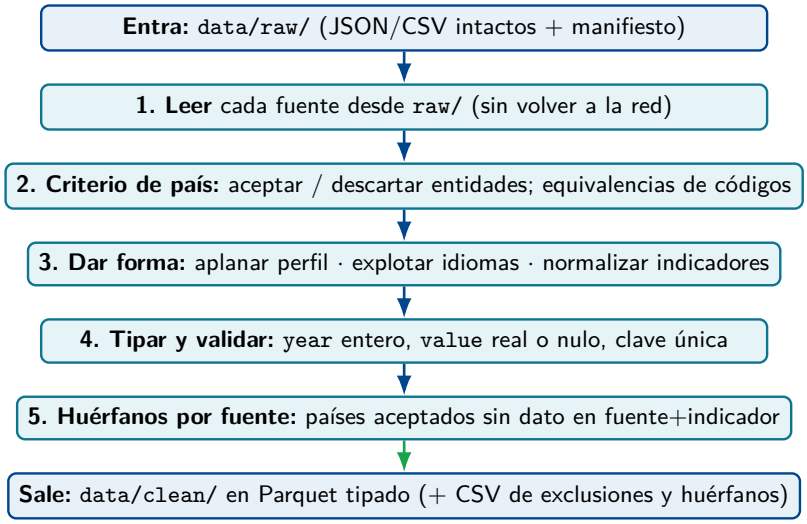


Figura 2: Flujo de `src/transform.py`: lectura de `raw/`, criterio, forma, tipado, huérfanos y escritura en `clean/`.

1. Leer.
Transform trabaja solo con lo que ya está en `raw/`. Si falta un fichero, el fallo es del extract (o de la idempotencia), no se "arregla" inventando filas aquí.

2. Criterio de país.
Ya no basta el `if iso3 not in focus_countries`. Decidís qué cuenta como país, escribís lo aceptado en `countries.parquet` y todo lo demás en `excluded_entities.csv` con motivo. Si una fuente usa otro código para la misma entidad, la equivalencia va en configuración.

3. Dar forma.
Lo que ya sabéis de la P1, pero para todo el universo: perfil aplanado, idiomas en tabla larga, indicadores de todas las fuentes (y toda la serie) al esquema común (`iso3`, `indicator_id`, `indicator_label`, `year`, `value`, `unit`, `source_id`).

4. Tipar y validar.
El año del Banco Mundial llega como texto: aquí se convierte a entero. Un valor ausente queda nulo, no cero. La clave (`iso3`, `indicator_id`, `year`, `source_id`) no puede duplicarse: si hay duplicado, error.

5. Huérfanos.
Por cada país aceptado, fuente e indicador: si no hay ninguna fila de datos, una línea en `join_orphans.csv`. Así medís cobertura; no asumís que todo casó.

La escritura a Parquet (y el particionado Hive de `indicators/`) puede vivir al final de Transform o en Load: lo que importa es que, cuando termine el pipeline, `data/clean/` quede como en el contrato. Spark y las fichas **no** deben leer `raw/`.

 CC BY-SA 4.0

Ayuda · por dónde empezar en transform

Abríd el `transform.py` de la Práctica 1 y localizad dónde se filtra por `focus_countries` y dónde se asume un solo año o un fichero por país. Esos son los puntos que el universo rompe. El diagrama de arriba es el orden mental: primero criterio y forma, después tipado y huérfanos, al final escribir `clean/`.

Los seis indicadores del Banco Mundial (ejemplos reales)

Obligatorio: **al menos seis** series del catálogo *World Development Indicators*, para **todos** los países y **todos** los años que ofrezca la API. Dos códigos son fijos; los otros cuatro los elegís vosotros y los justificáis en una línea en `ENTREGA_02.md`.

La URL de cada uno sigue el mismo patrón (sustituid el código):

```
https://api.worldbank.org/v2/country/all/indicator/NY.GDP.PCAP.CD?format=
    ↪ json
```

Catálogo para buscar más: https://data.worldbank.org/indicator

Código	**Qué mide (nombre oficial WDI)**	**Rol**
`NY.GDP.PCAP.CD`	GDP per capita (current US$). En español: PIB per cápita (USD corrientes).	**Fijo**
`SP.POP.TOTL`	Population, total. En español: población total.	**Fijo**
`SP.DYN.LE00.IN`	Life expectancy at birth, total (years). Esperanza de vida al nacer.	Ejemplo
`EN.ATM.CO2E.PC`	CO2 emissions (metric tons per capita). Emisiones de CO_2 per cápita.	Ejemplo
`IT.NET.USER.ZS`	Individuals using the Internet (% of population). Uso de Internet.	Ejemplo
`SP.URB.TOTL.IN.ZS`	Urban population (% of total population). Población urbana.	Ejemplo
`SL.UEM.TOTL.ZS`	Unemployment, total (% of total labor force). Desempleo.	Ejemplo
`NY.GDP.MKTP.KD.ZG`	GDP growth (annual %). Crecimiento anual del PIB.	Ejemplo
`SH.DYN.MORT`	Mortality rate, under-5 (per 1,000 live births). Mortalidad <5 años.	Ejemplo
`FP.CPI.TOTL.ZG`	Inflation, consumer prices (annual %). Inflación (IPC).	Ejemplo

 CC BY-SA 4.0

Ayuda · cómo no liaros con los seis

- "Seis" cuenta **códigos distintos** del Banco Mundial, no seis ficheros a mano ni seis países. Los dos fijos + cuatro elegidos = seis como mínimo.
- Podéis usar exactamente cuatro de la columna "Ejemplo", u otros del catálogo WDI: lo importante es que el código exista en la API y lo documentéis.
- Si en la Práctica 1 ya teníais PIB o población del BM, **no los contéis dos veces**: son los mismos códigos; aquí pasan a universo completo y serie completa.
- Las fuentes que no son Banco Mundial (Eurostat, OWID, perfil…) siguen siendo adicionales: no sustituyen a esos seis.

Mapa mental: de lo viejo a lo nuevo

La figura 3 separa lo que se conserva del repo de la Práctica 1, lo que se reescribe y lo que se crea de nuevo.

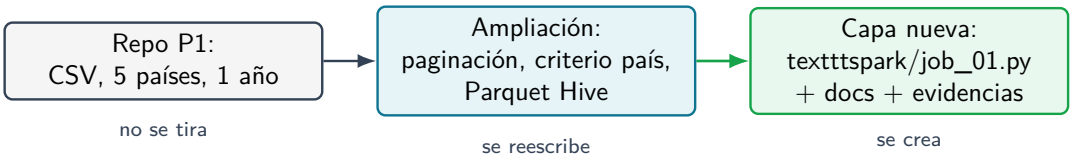


Figura 3: Mapa mental de la ampliación: lo que se conserva, lo que se reescribe y lo que se crea.

Teoría 1 · Por qué cinco países no bastan

Con cinco países y un año el pipeline vive en el caso feliz. Al pedir **todos** los países y **toda** la serie, el problema cambia de naturaleza: ya no es "más filas", es otro contrato con la realidad. La figura 4 resume ese contraste.

Un pipeline que solo se ha probado con el caso feliz no está probado. Además, el tipado deja de poder "verse a ojo": un año como texto (`"2023"`) frente a un año como entero (`2023`) rompe un join sin lanzar un error claro.

 CC BY-SA 4.0

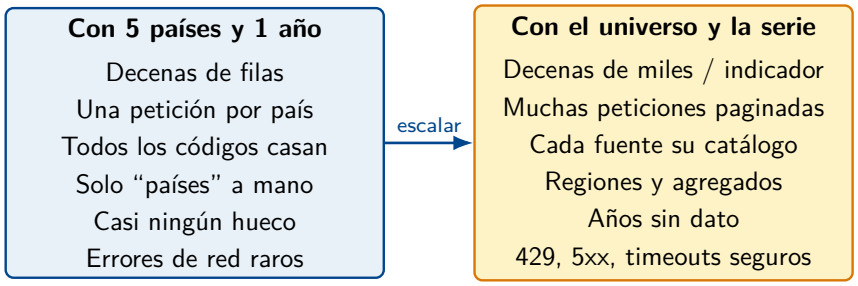


Figura 4: Contraste entre el caso de cinco países y un año y el universo con serie completa.

Ayuda · qué falla de verdad al escalar

Estos problemas **existen** en las fuentes que usáis (comprobados sobre la API del Banco Mundial y el perfil mledoze, octubre 2026): hay del orden de **79 agregados** en el catálogo WB (regiones, "Arab World", "European Union", . . .) frente a ∼217 entidades no agregadas; el perfil trae ∼250 entradas, de las cuales decenas no son estados independientes; un indicador del universo declara ∼17 556 registros (hay que paginar); el campo de año llega como **texto**; hay series con muchos nulos (p. ej. Kosovo: decenas de años sin PIB). El starter de la P1, con cinco países, os los oculta.

Teoría 2 · Paginación: la API os dice cuánto hay

Las respuestas del Banco Mundial empiezan con un bloque de metadatos. Ese bloque no es decoración: es el contrato de completitud, como muestra la figura 5.

```
{"page": 1, "pages": 8778,
 "per_page": 2, "total": 17556,
 "lastupdated": "2026-10-08"}
```

Metadatos reales (PIB pc, universo, `per_page=2`, 28/10/2026).

Vosotros debéis

1. Recorrer todas las páginas
2. Elegir un `per_page` sensato
3. Lo guardado = `total`
4. Si no cuadra: error

Figura 5: Metadatos de paginación del Banco Mundial y obligaciones del extract.

Qué es eso de la "predicción" de filas (y dónde va)

Hay **dos números distintos**. No los mezcléis. La figura 6 los pone lado a lado.

Por qué no vale "6 × 17 556" como predicción del limpio. Esa multiplicación solo estimaría el bruto de seis indicadores *antes* de filtrar. En el limpio no entran los

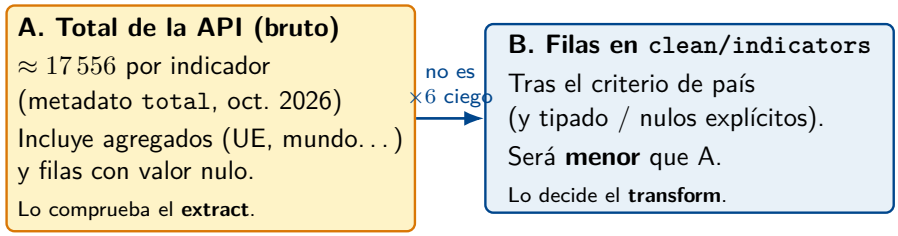


Figura 6: Dos cifras distintas: total bruto de la API (A) y filas en `clean/indicators` tras el criterio (B).

agregados que descartéis, ni las entidades que vuestro criterio rechace. Además, cada indicador puede tener un `total` distinto: no asumáis que los seis valen exactamente 17 556.

Dónde se escribe la predicción. En `ENTREGA_02.md`, **antes** de la primera ejecución completa, con un commit que lo demuestre. No va dentro de `transform.py` ni de `load.py`: es un razonamiento vuestro (por indicador y en total), no una función del pipeline.

Dónde se aplica el filtrado que hace caer el número. En `src/transform.py`: ahí leéis el bruto de `raw/`, aplicáis el criterio de país y escribís solo las filas aceptadas en `data/clean/indicators/`. `load.py` se limita a cargar ese limpio al SQLite (y lo que pida serving): **no** es el sitio donde decidís qué es un país ni cuántas filas de indicadores sobreviven.

Extract
Debe guardar el bruto completo: lo descargado por indicador debe cuadrar con el `total` de la API. Si no cuadra, el extract falla (no "aproximéis").

Transform
Decide qué filas pasan a `clean/indicators` (países aceptados, tipos, nulos explícitos, sin duplicados).

Load
Copia/materializa el limpio hacia `serving/`. No recalcula el criterio de país.

Documento
La predicción (cifras A y B razonadas) vive en `ENTREGA_02.md`.

 CC BY-SA 4.0

Ayuda · cómo razonar la predicción (sin código)

En el documento, en dos columnas mentales: (1) "espero ≈ T filas brutas por indicador porque el metadato `total` dice T"; (2) "tras el criterio, espero del orden de N entidades × Y años con dato (o con nulo explícito, si las guardáis)". No hace falta acertar al dígito: hace falta mostrar de dónde sale la cifra y, después, explicar la diferencia con lo obtenido. El commit de esa predicción tiene que ser **anterior** al commit donde ya corristeis el pipeline completo.

Teoría 3 · Extracción cortés e idempotente

Qué hacer con cada código HTTP

La figura 7 resume la decisión por tipo de respuesta.

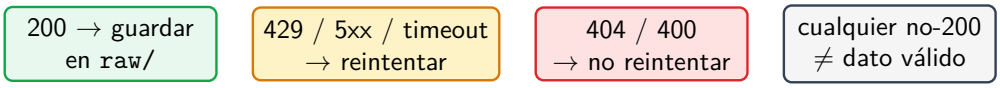


Figura 7: Qué hacer en el extract según el código HTTP de la respuesta.

Espera creciente típica: 1 s, 2 s, 4 s, 8 s... con un máximo de intentos. Cortesía: como máximo **cinco peticiones por segundo** contra la misma API, y un `User-Agent` que identifique al equipo.

Idempotencia

Definición · idempotencia (en el extract)

Una operación es **idempotente** si hacerla dos veces deja el mismo resultado útil que hacerla una. En el extract: si `data/raw/` ya tiene una respuesta buena (HTTP 200 guardada correctamente) para una URL, **volver a ejecutar el pipeline no debe volver a descargarla**, salvo que pidáis explícitamente forzar una re-descarga. Los ficheros buenos se reutilizan; el manifiesto puede ganar filas de auditoría (nuevo `run_id`), pero no debe reescribirse borrando el historial.

Imaginad que la primera ejecución ya bajó las seis series del Banco Mundial (varias páginas cada una). La segunda ejecución, minutos después, no necesita martillear otra vez la API con las mismas peticiones: el dato ya está en disco. Eso ahorra tiempo, evita 429 y demuestra que el extract "recuerda" lo hecho (figura 8).

 CC BY-SA 4.0

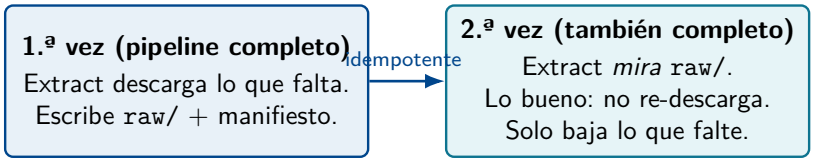


Figura 8: Primera y segunda ejecución del pipeline cuando el extract es idempotente.

Qué no es idempotencia (confusión habitual)

En la Práctica 1 el pipeline permite **saltarse el extract** para depurar solo transform o load (no volver a tocar la red). Eso es útil, pero es otra cosa, como aclara la figura 9:

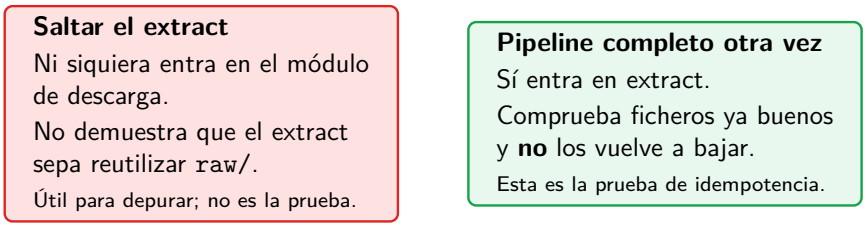


Figura 9: Diferencia entre saltarse el extract (depuración) e idempotencia real.

Resumen: saltar el extract = "hoy no miro la descarga". Idempotencia = "sí miro la descarga, y como ya está bien, no repito el trabajo".

Qué sí debe pasar en la 2.ª ejecución

1. Lanzáis el pipeline **completo** (extract incluido), sin pedir re-descarga forzada.
2. En terminal / manifiesto: no aparecen descargas nuevas de URLs que ya tenían un 200 válido en `raw/`.
3. El contenido útil de `raw/` (los JSON buenos) queda igual.
4. El manifiesto es **acumulativo**: se añaden filas del nuevo `run_id`; no se borra lo anterior.
5. Transform y load pueden regenerar `clean/` y `serving/` desde ese `raw/` ya existente.

Si necesitáis bajar otra vez (API actualizada, fichero corrupto), eso es una **re-descarga forzada** explícita: la excepción controlada a la idempotencia, no el comportamiento por defecto.

 CC BY-SA 4.0

Ayuda · mirad el extract de la Práctica 1

Sin enseñaros la solución: en el starter, ¿vuelve a descargar siempre? ¿El manifiesto se *añade* o se *sobrescribe*? ¿Trata un no-200 como dato? Esas preguntas os dicen qué falta para ser idempotentes al escalar. Con seis indicadores y paginación real, la segunda ejecución idempotente debería tardar mucho menos en red que la primera: si tarda igual, probablemente estáis re-descargando todo.

Teoría 4 · ¿Qué es un país?

El perfil (mledoze/countries) trae ∼250 entradas: países, territorios, dependencias. Las APIs de indicadores añaden agregados (regiones, grupos de renta) que **no** debéis tratar como país si luego calculáis medianas regionales. El flujo del criterio aparece en la figura 10.

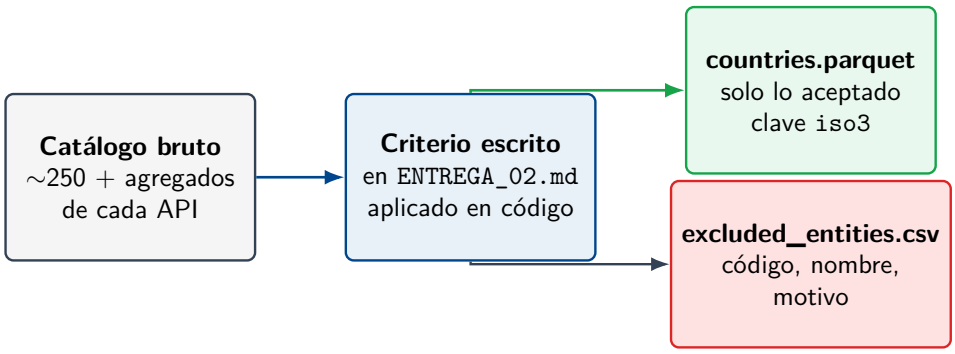


Figura 10: Aplicación del criterio de país: aceptados en Parquet y descartados con motivo.

No hay respuesta única. Sí hay respuestas defendibles: criterio escrito, aplicado por código, y nada descartado en silencio. Si una entidad aceptada tiene códigos distintos entre fuentes, la equivalencia vive en configuración.

Si un agregado se cuela como país, la mediana regional de las fichas queda distorsionada. Ese es justo el tipo de defecto que el universo destapa.

 CC BY-SA 4.0

Ayuda · criterio defendible

Dos pistas en los propios datos (no son la única respuesta válida):

- En el catálogo de países del Banco Mundial, los agregados vienen etiquetados como región *Aggregates* (decenas de entradas: `AFE`, `EUU`, `WLD`, ...).
- En el perfil mledoze, el campo `independent` distingue estados de territorios y dependencias.

Probad mentalmente: si tratáis la "European Union" como un país más de Europa, la mediana del PIB regional **se mueve** (es un efecto real, no teórico). Documentad el criterio y listad en `excluded_entities.csv` todo lo que caiga fuera, con motivo.

Teoría 5 · De fila a columna: por qué Parquet

Definición · Parquet

Parquet es un formato de fichero *columnar* (orientado a columnas) para almacenar tablas en disco. Frente a un CSV, que guarda cada fila entera seguida de la siguiente, Parquet agrupa juntos los valores de cada columna. Eso permite: (1) leer solo las columnas que hacen falta, (2) comprimir mejor (valores parecidos van juntos) y (3) llevar el **esquema y los tipos** dentro del propio fichero (enteros, reales, nulos), no como texto suelto. Es el formato habitual en pipelines de datos y en herramientas como Spark; en esta práctica sustituye al CSV en la zona `clean/`.

Definición · zona limpia en Parquet tipado

La **zona limpia** es la carpeta `data/clean/`: ahí viven las tablas *después* del Transform y *antes* de cargar el SQLite de servicio. "En Parquet tipado" significa que esas tablas ya no se guardan como CSV de texto, sino como ficheros Parquet donde `year` es entero, `value` es real (o nulo), e `iso3` es texto, según el contrato de columnas. Sí: **va dentro de `data/`**, concretamente en `data/clean/`, junto a (y no dentro de) `data/raw/` y `data/serving/`.

Qué quiere decir eso, en orden

Recordad las tres zonas bajo `data/` de la Práctica 1. No han cambiado de sitio; ha cambiado *cómo* se materializa la del medio (figura 11).

 CC BY-SA 4.0

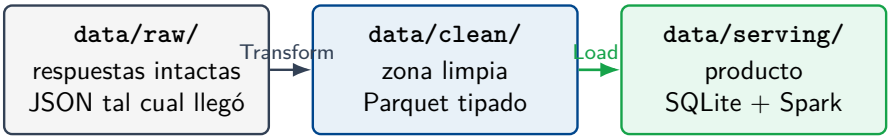


Figura 11: Las tres zonas bajo `data/`: bruto, limpio tipado y servicio.

Zona.
No es un concepto abstracto: es un **directorio en disco**. La zona limpia es `data/clean/`. Si alguien guarda el Parquet en `cards/` o en la raíz del repo, ha roto el contrato del pipeline: el Load y Spark deben saber dónde leer.

Limpia.
"Limpio" no significa "bonito". Significa que ya pasó por Transform: columnas con nombres estables, `iso3` como clave, indicadores con el mismo esquema, nulos marcados, sin el JSON anidado de cada API. Es lo que se puede unir y consultar sin pelearse con el formato bruto.

Parquet.
El formato del fichero (véase la definición anterior). En la Práctica 1 podíais dejar CSV aquí; en esta práctica el limpio de países, idiomas e indicadores va en Parquet.

Tipado.
El fichero declara tipos de verdad: el año no es la cadena `"2023"` disfrazada de número; es un entero. El valor ausente es un nulo, no un cero inventado. Eso es lo que evita joins silenciosos entre fuentes.

Particionado (en indicadores).
Además, `data/clean/indicators/` se parte en subcarpetas `indicator_id=.../`. Eso sigue siendo zona limpia: solo organiza el Parquet para no abrir ficheros que no hacen falta.

En resumen: cuando el enunciado pide "zona limpia en Parquet tipado", os está pidiendo que, tras el Transform, escribáis en `data/clean/` (sí, bajo `data/`) ficheros Parquet con tipos correctos, no un CSV improvisado ni un volcado en otra carpeta.

Ayuda · no confundir clean con serving

`data/clean/` es el almacén limpio intermedio. `data/serving/` es lo que consume el producto (SQLite, salida Spark, ...). Las fichas de `cards/` se generan desde serving (o, si aún depuráis, desde clean), **nunca** desde `raw/`. Los tres viven bajo `data/` porque son datos regenerables: no se suben al GitHub.

 CC BY-SA 4.0

CSV frente a Parquet

En CSV los datos van **fila a fila**. Para leer solo la columna `value` hay que atravesar todo el fichero. En Parquet los datos van **columna a columna**: se lee solo lo pedido, valores parecidos comprimen mejor, y el esquema viaja con el fichero. La figura 12 lo dibuja con un color por fila frente a un color por columna.

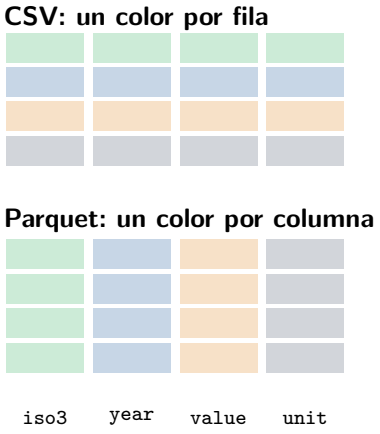


Figura 12: Organización de datos en CSV (por filas) frente a Parquet (por columnas).

Dónde se cambia en vuestro código.

En el starter de la Práctica 1, `transform.py` escribe CSV en `data/clean/` y `load.py` los lee con `pd.read_csv` hacia SQLite. Eso es lo que tenéis que evolucionar:

Fichero	**Qué cambia respecto a la P1**
`src/transform.py`	Al final del Transform, en lugar de (o además de) CSV, escribid Parquet tipado en `data/clean/`. Ahí nace `countries.parquet`, `languages.parquet` y el árbol `indicators/indicator_id=.../`.
`src/load.py`	Dejad de depender de `*.csv` en `clean/`: leed los Parquet y cargad SQLite en `data/serving/`. El comparativo de fichas sigue saliendo del SQLite, no del JSON bruto.
`requirements.txt`	Añadid `pyarrow` (pandas lo usa al hacer `to_parquet`/`read_parquet`). Sin esa dependencia el clon limpio no regenera.
`spark/job_01.py`	**No** sustituye a Transform: solo *lee* el Parquet de `clean/indicators/` ya generado.

 CC BY-SA 4.0

Podéis concentrar la escritura Parquet al final de Transform (recomendado: el contrato de limpio queda cerrado ahí) o en Load; lo que no vale es dejar el limpio obligatorio solo en CSV.

Ejemplos mínimos (orientativos; no son la entrega).

Los scripts `01_csv_a_parquet.py`, `02_leer_parquet.py` y `09_csv_vs_parquet_tamano.py` de `practica02_prototipos/` (Prototipos) cubren el mismo flujo sobre el CSV mini. Con un `DataFrame` de pandas ya tipado tras la limpieza, la escritura queda así:

Listing 1: Escribir una tabla limpia en Parquet

```
# Idea para el final de transform.py (o un helper que llame)
# df_countries: columnas iso3, name_common, region, ...
df_countries.to_parquet(
    "data/clean/countries.parquet",
    index=False,
)
```

Listing 2: Leer Parquet en load.py hacia SQLite

```
# Idea: sustituir pd.read_csv(...) por lectura Parquet
df = pd.read_parquet("data/clean/countries.parquet")
df.to_sql("countries", conn, index=False, if_exists="replace")
```

Para indicadores particionados al estilo Hive, pandas/pyarrow pueden escribir el directorio de una vez (la columna de partición no hace falta duplicarla dentro del fichero si usáis particionado):

Listing 3: Escribir indicators particionado (idea)

```
# df_ind tiene indicator_id, iso3, year, value, ...
df_ind.to_parquet(
    "data/clean/indicators",
    index=False,
    partition_cols=["indicator_id"],
)
```

Y para *consultar* sin cargar todo a memoria (útil al comprobar el contrato o al preparar evidencia):

Listing 4: Consultar el dataset particionado con pyarrow

```
import pyarrow.dataset as ds
```

 CC BY-SA 4.0

```
d = ds.dataset(
    "data/clean/indicators",
    format="parquet",
    partitioning="hive",
)
print(d.count_rows())
print(d.schema)
# Filtro que poda particiones: solo un indicador
t = d.to_table(filter=(ds.field("indicator_id") == "NY.GDP.PCAP.CD"))
print(t.num_rows)
```

Esa consulta es la que se usa al clonar para contar filas y revisar el esquema. El prototipo `04_consulta_pyarrow.py` la aplica al mini Hive; `spark/job_01.py` hará un `spark.read.parquet(...)` equivalente sobre el mismo directorio.

Ayuda · tipado al escribir

Antes de `to_parquet`, aseguraos de los dtypes: por ejemplo `year` como entero y `value` como float con nulos reales (`NA`), no como texto. Si el Parquet guarda el año como string, el "tipado" del enunciado no se cumple aunque la extensión sea `.parquet`.

Anatomía de un fichero Parquet

La figura 13 sitúa row groups, chunks y footer.

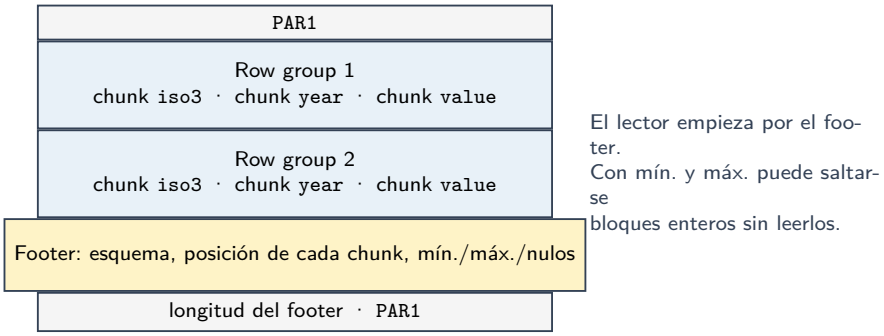


Figura 13: Anatomía de un fichero Parquet: row groups, column chunks y footer.

 CC BY-SA 4.0

Particionado estilo Hive

Definición · particionado estilo Hive

Particionar consiste en guardar una misma tabla lógica en **varias carpetas**, una por valor de una columna. En el estilo Hive el valor figura en el *nombre del directorio* (`indicator_id=NY.GDP.PCAP.CD/`) y no es necesario repetirlo en cada fila del fichero. Si el filtro usa esa columna, se puede **abrir solo esa carpeta** y omitir las demás (poda de particiones).

En el observatorio, la tabla limpia de indicadores tiene filas como estas (país + año + indicador + valor):

iso3	year	indicator_id	value
ESP	2020	NY.GDP.PCAP.CD	27056.2
ESP	2021	NY.GDP.PCAP.CD	30103.5
FRA	2020	NY.GDP.PCAP.CD	39030.1
ESP	2020	SP.POP.TOTL	47351567
FRA	2020	SP.POP.TOTL	67571162
DEU	2021	SP.DYN.LE00.IN	80.9

Es **una** tabla lógica. En disco, con partición por `indicator_id`, esas filas se *reparten* en carpetas: todas las de PIB juntas, todas las de población juntas, etc. País y año siguen **dentro** de cada Parquet; lo que organiza carpetas es solo el indicador. La figura 14 lo muestra.

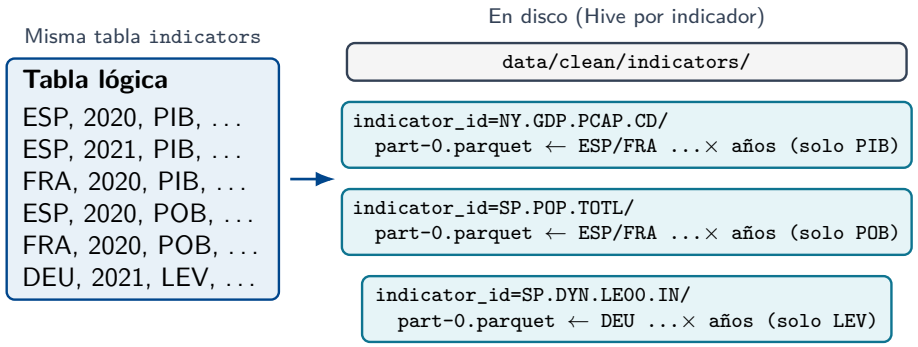


Figura 14: De la tabla país–año–indicador a carpetas Hive por `indicator_id`. Dentro de cada Parquet siguen estando `iso3` y `year`. El prototipo `03_particionado_hive.py` genera ese árbol en `out/indicators_hive/`.

 CC BY-SA 4.0

Efecto en una consulta.

Ante un filtro del tipo "PIB per cápita de España en toda la serie", el lector se limita a `indicator_id=NY.GDP.PCAP.CD/` y aplica `iso3=ESP` dentro de ese fichero. No se abren las carpetas de población ni de esperanza de vida. Esa **poda** reduce disco leído y trabajo en Spark.

Lo que pedimos en esta práctica.

Particionad `data/clean/indicators/` por `indicator_id` (unos pocos directorios: los ≥ 6 del Banco Mundial). **No** particionéis por `iso3`: saldrían cientos de carpetas diminutas (una por país), cada una con trocitos de todos los indicadores. La figura 15 contrasta ambas ideas.

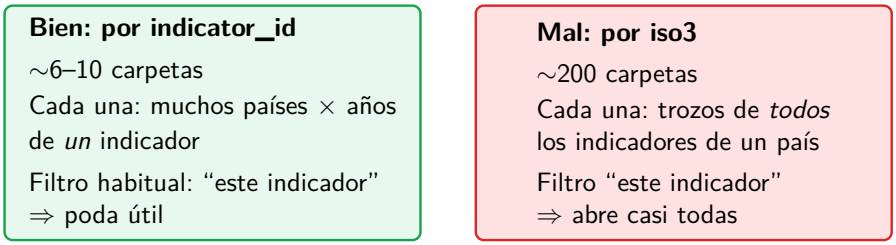


Figura 15: Misma tabla país–año–indicador: particionar por indicador (recomendado) frente a particionar por país (evitar).

Cómo se elige la columna.

Se elige la columna que aparece en el filtro habitual y que tiene **pocos valores distintos**: así se abren pocas carpetas y cada una agrupa muchas filas. En un catálogo de discos, si la pregunta recurrente es el género (rock, jazz,...), se ordena por género (∼10 estanterías). Si se ordenara por año de edición (∼70 estanterías), una búsqueda por género obligaría a recorrer casi todas. El mismo criterio se aplica al observatorio:

	Catálogo (analogía)	**Observatorio**
Filtro habitual	género	indicador (p. ej. PIB)
Buena partición	género (∼10)	`indicator_id` (∼6–10)
Mala partición	año (∼70)	`iso3` (∼200)
Dentro de cada carpeta	muchos discos	muchos países × años

En esta práctica la columna de partición es `indicator_id`. Particionar por `iso3` multiplica carpetas diminutas y anula la poda cuando el filtro es el indicador.

 CC BY-SA 4.0

Teoría 6 · Tipado: el contrato que el CSV no impone

Columna	**Tipo**	**Regla**
`iso3`	texto	En `countries`; nunca un agregado
`indicator_id`	texto	Columna de partición
`indicator_label`	texto	Etiqueta legible
`year`	entero	Igual en todas las fuentes; nunca texto
`value`	real	Nulo si falta; nunca un cero inventado
`unit`	texto	De `sources.yaml`
`source_id`	texto	Origen de la fila

Clave única: (`iso3`, `indicator_id`, `year`, `source_id`). Duplicado = error, no aviso. Huérfanos **por fuente**: cada país aceptado sin ningún dato en esa fuente e indicador, listado en `join_orphans.csv`. La figura 16 resume tres fallos típicos si el tipado falla.

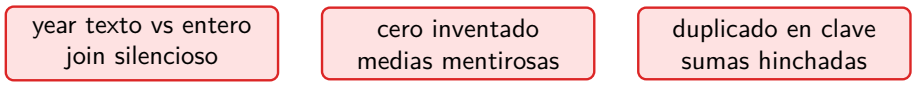


Figura 16: Tres riesgos de tipado: año texto, ceros inventados y duplicados de clave.

Ayuda · tipado y nulos

En el JSON del Banco Mundial el año viaja como cadena (`"2025"`, no como número). Si una fuente lo deja texto y otra lo casta a entero, un cruce por año puede quedar vacío **sin error**. Los nulos tampoco son teóricos: hay países con gran parte de la serie sin valor. Un cero inventado para "rellenar" cambia medias y medianas; dejad el nulo explícito y que la ficha diga que falta el dato.

Teoría 7 · MapReduce

Definición · MapReduce

MapReduce es un modelo de agregación sobre muchas filas en tres fases: (1) cada fila se procesa por separado y emite un par clave–valor (**Map**); (2) los pares con la misma clave se reúnen (**Shuffle**); (3) se combinan los valores de cada clave (**Reduce**). No se exige que una sola máquina materialice todas las filas a la vez.

 CC BY-SA 4.0

Analogía: recuento de votos.

En un colegio electoral, cada papeleta se anota como "partido X → 1" (Map). Las papeletas del mismo partido se agrupan en la misma mesa (Shuffle). En cada mesa se suman los "1" (Reduce). El total por partido se obtiene sin acumular todas las papeletas en una sola mano.

Aplicación al observatorio.

La pregunta de cobertura es: *cuántas filas con valor no nulo hay por indicador y por década* (p. ej. PIB en los 80, población en los 2000). Se parte de filas limpias como estas:

País	Indicador	Año
ESP	PIB	1987
FRA	PIB	1983
ESP	POB	2004

Map. Cada fila emite "(indicador, década) → 1". El país no interviene en esta agregación; el año 1987 se proyecta a la década 1980s y el 2004 a 2000s:

Fila	Emisión del Map
ESP, PIB, 1987	(PIB, 1980s) → 1
FRA, PIB, 1983	(PIB, 1980s) → 1
ESP, POB, 2004	(POB, 2000s) → 1

Shuffle. Las emisiones (PIB, 1980s) se reúnen en la lista $[1, 1]$; la de población, en $[1]$. Esta fase mueve datos y suele ser la más costosa.

Reduce. Se obtiene (PIB, 1980s) → 2 y (POB, 2000s) → 1: la cobertura por indicador y década.

La figura 17 resume ese recorrido.

- **Map:** se emite un par clave–valor por fila; las filas se pueden tratar en paralelo.
- **Shuffle:** se reúnen los pares con la misma clave; es la fase que más mueve datos.
- **Reduce:** se combinan los valores de cada clave (aquí, suma de los "1").

Dónde se usa en esta práctica.

No se pide programar Hadoop. La agregación de **cobertura por (`indicator_id`, década)** se implementa en `spark/job_01.py` tres veces (RDD con `map`/`reduceByKey`,

 CC BY-SA 4.0

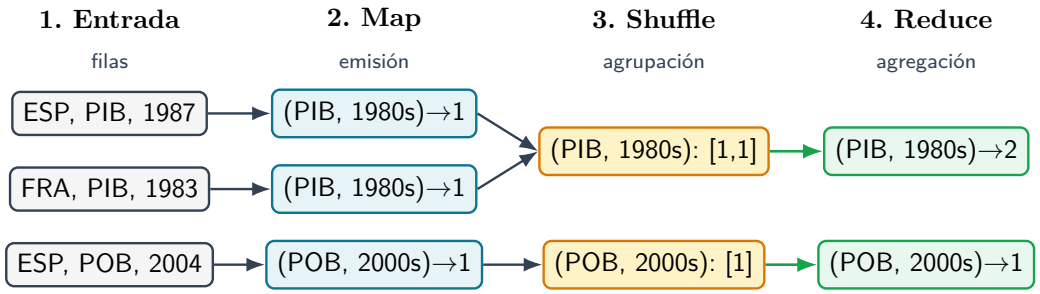


Figura 17: MapReduce: filas, emisión, agrupación y agregación (cobertura por indicador y década).

DataFrame con `groupBy`, Spark SQL) y debe coincidir entre sí y con un `COUNT` en SQLite. La salida se escribe en `data/serving/spark/cobertura_decada/`. En la Teoría 8 se ve que Spark aplica el mismo modelo aunque DataFrame y SQL no nombren "MapReduce" explícitamente.

El recorrido se ensaya antes en `05_mapreduce_puro.py`, `07_spark_mapreduce_rdd.py` y `08_spark_dataframe_sql.py` (Prototipos).

Teoría 8 · Spark: el mismo modelo en vuestro portátil

Definición · Apache Spark

Apache Spark es un motor de procesamiento de datos a gran escala. El trabajo se reparte en **tareas** paralelas sobre **particiones** de los datos. Se distingue un *driver* (construye el plan y coordina) de los *executors* (ejecutan el cálculo). Sobre el mismo motor se ofrecen varias APIs (RDD, DataFrame, Spark SQL). No es una base de datos ni sustituye a SQLite: lee Parquet (u otros formatos), transforma y escribe resultados. En esta práctica se usa en modo local (`local[*]`) en el portátil; el modelo de programación es el del clúster, aunque aquí todo corre en un solo proceso.

Driver y executors

La figura 18 sitúa driver y executors en el portátil.

En un clúster, driver y executors son máquinas distintas. En `local[*]` todo vive en un proceso: el modelo de programación es el mismo; lo que aprendéis aquí escala.

 CC BY-SA 4.0

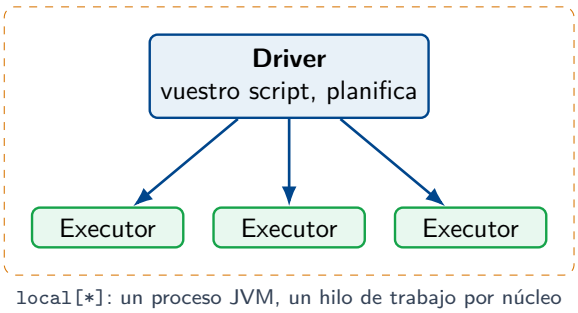


Figura 18: Driver y executors en modo `local[*]` en un solo proceso.

Tres APIs para la misma cuenta

API	Cómo se piensa	Optimizador
RDD	Objetos distribuidos; `map`, `reduceByKey`	Funciones opacas: las ejecuta tal cual
DataFrame	Tabla con esquema; `groupBy`, `agg`	Catalyst puede reordenar y podar
Spark SQL	SQL sobre vista temporal	Lo mismo que DataFrame: mismo plan

En el job escribís las tres y comprobáis que dan lo mismo (y que DataFrame coincide con SQLite).

Transformaciones, acciones y pereza

La figura 19 distingue lo que solo describe trabajo de lo que lo dispara.

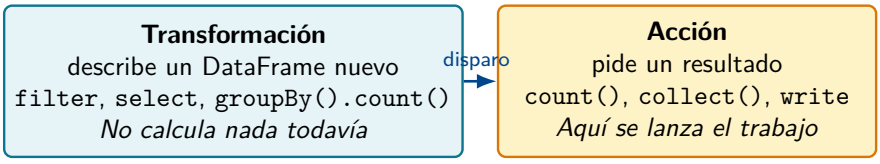


Figura 19: Transformación perezosa frente a acción que dispara el trabajo.

Ojo: `df.groupBy("x").count()` es transformación (devuelve un DataFrame); `df.count()` es acción (devuelve un número).

Dependencia **estrecha**: cada partición de salida depende de una de entrada (`filter`, `select`): sin movimiento de datos. Dependencia **ancha**: una partición de salida necesita muchas de entrada (`groupBy`, `join`): hay *shuffle*.

 CC BY-SA 4.0

Jobs, stages, tareas y particiones

La jerarquía de ejecución se resume en la figura 20.

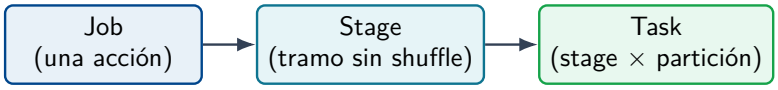


Figura 20: Jerarquía de ejecución en Spark: job, stage y task.

Al leer, el número de particiones depende del número y tamaño de ficheros y de `spark.sql.files.maxPartitionBytes` (128 MB por defecto). Tras un shuffle, de `spark.sql.shuffle.partitions` (200 por defecto), que AQE puede agrupar. `df.rdd.getNumPartitions()` os lo dice. Con `local[1]` y `local[4]` el número de particiones *al leer* normalmente **no** cambia: cambia el paralelismo al ejecutar.

La Spark UI (`http://localhost:4040`) muere al terminar el programa. Para conservar evidencia: `spark.eventLog.enabled`. Spark 4.2, con el log activado, guarda un subdirectorio comprimido (zstd) bajo el directorio que indiquéis.

Arranque de sesión (único fragmento de código que se os da)

```
from pathlib import Path
from pyspark.sql import SparkSession

logs = Path("logs/spark-events").resolve()
logs.mkdir(parents=True, exist_ok=True)

spark = (
    SparkSession.builder
    .appName("job01")
    .master("local[*]")
    .config("spark.eventLog.enabled", "true")
    .config("spark.eventLog.dir", logs.as_uri())
    .getOrCreate()
)
```

Se requiere Java 17, 21 o 25 y Python 3.10+ (Spark 4.2). Hay que añadir `pyspark` a `requirements.txt`. PySpark 4.2 aún no soporta del todo pandas 3. El arranque mínimo se reproduce en `06_spark_session.py` (Prototipos).

 CC BY-SA 4.0

Instalar Java para Spark (Linux y Windows)

PySpark no sustituye a la JVM: Spark arranca un proceso Java. Sin un JDK 17, 21 o 25 visible en el `PATH`, el job falla con un muro de texto. Java 8 u 11 **no** valen para Spark 4.2. Comprobación en ambos sistemas:

```
java -version
# Debe mostrar algo como: openjdk version "21...."  (o 17 / 25)
```

Linux (Debian / Ubuntu y derivadas).

Se instala el JDK de OpenJDK desde los repositorios y se deja como predeterminado:

```
sudo apt update
sudo apt install -y openjdk-21-jdk
sudo update-alternatives --config java   # si hay varias versiones
java -version
```

En Fedora / RHEL: `sudo dnf install java-21-openjdk-devel`. Si `java -version` ya muestra 17/21/25, no hace falta reinstalar.

Windows.

Se recomienda un JDK Temurin (Eclipse Adoptium), LTS 21:

1. Abrir https://adoptium.net/ y descargar el instalador **.msi** de Temurin 21 (x64) para Windows.
2. En el asistente, marcar la opción que añade Java al `PATH` y, si aparece, la que define `JAVA_HOME`.
3. Cerrar y abrir un terminal **nuevo** (PowerShell o `cmd`) y ejecutar `java -version`.

Si `java` no se reconoce, se define la variable de usuario `JAVA_HOME` apuntando a la carpeta del JDK (la que contiene `bin\java.exe`) y se añade `%JAVA_HOME%\bin` al `PATH` (Configuración → Sistema → Acerca de → Configuración avanzada del sistema → Variables de entorno). Después, terminal nuevo y `java -version`.

Ayuda · si Spark no arranca

Orden de comprobación: (1) `java -version` en 17/21/25; (2) el mismo terminal donde activáis el `.venv`; (3) `pip show pyspark`; (4) probar `06_spark_session.py` del kit de prototipos antes del job del repo. Un Java viejo (8/11) o un `PATH` que apunta a otro `java.exe` es la causa más frecuente.

 CC BY-SA 4.0

Por qué Spark si el dataset aún cabe en memoria

Cabe de sobra; pandas lo resolvería, probablemente más rápido. Spark paga un coste fijo (JVM, plan, coordinación). Lo usáis para aprender el modelo con datos que podéis comprobar. En la práctica siguiente mediréis ese coste frente a pandas con más volumen (McSherry, Isard y Murray, 2015: COST).

Ayuda · fichas, Spark y el starter

Tres sitios donde el universo destapa defectos del código de la P1 (buscadlos vosotros; no os damos el parche):

- El comparativo regional de las fichas del starter **no fija el mismo año** entre el país y sus vecinos: con un solo año no se notaba; con toda la serie, mezcla décadas. Aquí pedimos mediana del mismo indicador y del mismo año.
- Antes de vuestra primera acción en el job, mirad el event log: a veces Spark ya ha registrado trabajo de arranque. Contestad con lo que veáis, no con lo que "debería" pasar.
- El número de particiones al *leer* Parquet suele depender de ficheros y de `maxPartitionBytes`, no de `local[1]` frente a `local[4]`. Comprobadlo; esa es la respuesta de la pregunta de teoría.

Qué tenéis que conseguir en código

Con la teoría anterior, el trabajo concreto sobre el repo de la Práctica 1 es este.

1. **Universo.** Criterio de país; `countries.parquet`; `excluded_entities.csv`; equivalencias en configuración.
2. **Extract a escala.** ≥ 6 indicadores BM (fijos `NY.GDP.PCAP.CD` y `SP.POP.TOTL`; cuatro más, p. ej. los de la tabla de ejemplos), serie completa, paginación cuadrada con `total`, reintentos, idempotencia, manifiesto acumulativo, cortesía.
3. **Transform con contrato.** Esquema tipado; clave única; huérfanos por fuente.
4. **Parquet particionado.** `indicators/` por `indicator_id`; `countries` y `languages` en Parquet; dependencias en `requirements.txt`.
5. **Serving y fichas.** SQLite completo; una ficha por país; mediana regional mismo indicador y mismo año.
6. **Job Spark.** `spark/job_01.py`: cobertura por (`indicator_id`, década) en RDD, DataFrame y SQL; coincidencia entre sí y con SQLite; plan; particiones; event log; salida en `data/serving/spark/cobertura_decada/`.

 CC BY-SA 4.0

7. **Bugs.** Al menos tres defectos que el universo destapa, documentados en `BUGS.md`.

Ayuda · ideas para BUGS.md (sin spoilear el arreglo)

Candidatos reales a documentar si os aparecen (reproducción + efecto + arreglo): respuesta no-200 tratada como dato; manifiesto que borra la historia al reejecutar; descarga repetida aunque el fichero ya era bueno; agregado colado en `countries` que sesga la mediana; año texto frente a entero; comparativo de ficha mezclando años; huérfanos que solo aparecían con cinco países y ahora explotan; partición mal elegida (cientos de ficheros diminutos). No hace falta sufrirlos todos: al menos tres, con evidencia vuestra.

Calendario y qué hay que entregar

La Práctica 2 se cierra en **dos hitos** en el mismo GitHub del equipo. En ambos días hay **preguntas en aula**. Si el clon falla o el código no regenera, se abrirán **issues** en cada repositorio con lo que haya que corregir. La figura 21 (página horizontal) resume la mini-rúbrica de lo que se comprueba.

 CC BY-SA 4.0

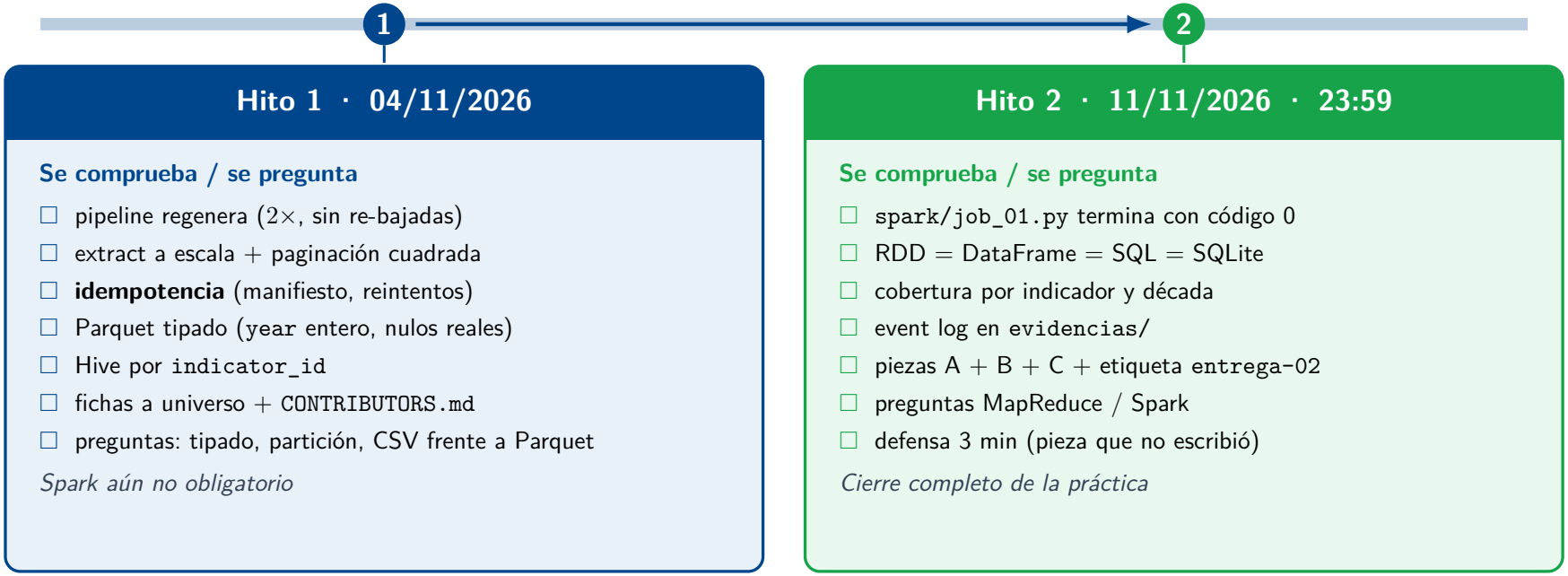


Figura 21: Mini-rúbrica de aula (página horizontal): el 04/11 se comprueba extract idempotente, Parquet/Hive y tipado; el 11/11, Spark/MapReduce y el cierre. Preguntas en ambos días.

Hito 1 (04/11) · Extract y Parquet.

Ese día se pide lo de la columna izquierda de la figura 21: extract a escala con **idempotencia**, transform tipado, Parquet Hive, fichas, `CONTRIBUTORS.md`. En aula se pregunta y se comprueba en el repo. El job Spark **aún no** es obligatorio.

Hito 2 (11/11) · MapReduce, Spark y cierre.

Límite **11/11/2026, 23:59**. Se pide lo de la columna derecha: `spark/job_01.py`, coincidencia RDD/DataFrame/SQL/SQLite, evidencias, piezas A–C, etiqueta `entrega-02`, preguntas y defensa.

Hay que entregar **tres piezas** (**A**, **B** y **C**) con esa etiqueta. Si falta alguna en el hito 2, la entrega no está completa.

A · Código ampliado y regenerable

Pipeline del repo P1 ampliado según "Qué tenéis que añadir". **04/11:** extract + transform + Parquet/Hive + fichas. **11/11:** además `spark/job_01.py`.
Desde un clon limpio de `entrega-02` debe bastar:

```
git clone <vuestra-url-de-la-practica-1> obs && cd obs
git checkout entrega-02
python3 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
python -m src.pipeline
python -m src.pipeline          # 2.a vez: sin re-descargas
python spark/job_01.py; echo "exit=$?"
```

Si el repo no es accesible o no regenera, la nota máxima es 4.

 CC BY-SA 4.0

B · Documentación (cierre 11/11)

En la raíz del repo:

- `CONTRIBUTORS.md`: todos los integrantes del equipo. Por cada persona, en una línea: **Nombre Apellidos** y **email institucional UCJC** de la facultad (no Gmail ni cuentas personales). Ejemplo de forma:

```
# Equipo observatorio - Practica 2
Ana Garcia Lopez - ana.garcia@alumno.ucjc.edu
Luis Perez Ruiz - luis.perez@alumno.ucjc.edu
```

- `ENTREGA_02.md` (1–2 páginas): predicción **commiteada antes** de ejecutar; resultado y diferencia; decisiones (criterio de país, indicadores, particionado); salidas de terminal; tres preguntas de teoría con *vuestros* datos; pendientes.
- `BUGS.md`: al menos tres defectos (reproducción, efecto, arreglo).

C · Evidencia de ejecución (cierre 11/11)

En `evidencias/`, el subdirectorio del event log del job Spark citado en el documento (Spark 4.2 / zstd: copiar el subdirectorio entero). En git, el commit de la predicción *antes* de la primera ejecución completa. No subir `data/` ni `logs/` completos.

Defensa (11/11).

Al inicio de la sesión: un miembro al azar, 3 minutos, enseña la entrega funcionando y explica una pieza que **no** haya escrito él. Si no puede, su nota se separa de la del equipo. Cada día de retraso resta 1 punto. Inventar cifras o hardcodear fichas suspende la práctica.

Referencias

- J. Dean y S. Ghemawat (2004). *MapReduce: Simplified Data Processing on Large Clusters*. OSDI 2004.
- M. Zaharia et al. (2012). *Resilient Distributed Datasets*. NSDI 2012.
- F. McSherry, M. Isard y D. G. Murray (2015). *Scalability! But at What COST?* HotOS XV.
- B. Chambers y M. Zaharia (2018). *Spark: The Definitive Guide*. O'Reilly. Partes I y II.
- M. Kleppmann (2017). *Designing Data-Intensive Applications*. O'Reilly. Cap. 10.

 CC BY-SA 4.0

- Apache Parquet: https://parquet.apache.org/docs/file-format/
- Apache Spark 4.2.0: https://spark.apache.org/docs/latest/
- World Bank API: https://datahelpdesk.worldbank.org/knowledgebase/articles/898581

 CC BY-SA 4.0